

IEEE-CIS Fraud Detection

Complete Analysis & Model Development

Project Overview & Agenda



What We'll Cover:

1. Dataset Understanding & EDA
2. Data Preprocessing
3. Model Building (5 Models)
4. Model Evaluation & Comparison
5. Feature Importance Analysis
6. Results



Goal: Detect fraudulent credit card transactions with ML

Step 1: Dataset Understanding (Original Dataset)

590,540

Transactions

394

Features

3.5%

Fraud Rate



Feature Categories:

- TransactionAmt, TransactionDT (transaction details)
- card1-card6 (card information)
- addr1, addr2 (address data)
- dist1, dist2 (distance metrics)
- P_emaildomain, R_emaildomain (email domains)
- C1-C14 (count features)
- D1-D15 (timedelta features)
- M1-M9 (match features)
- V1-V339 (Vesta engineered features)

Step 1: Dataset Understanding (Stratified sample)

100,000

Transactions

394

Features

3.5%

Fraud Rate

⚠ Original dataset is large and memory intensive, we apply stratified sampling to reduce size while preserving class distribution

- Target Variable (isFraud)
- Genuine (0): 96500 (96.50%)
- Fraudulent (1): 3499 (3.50%)
- Imbalance Ratio: 27.6:1

Data types:

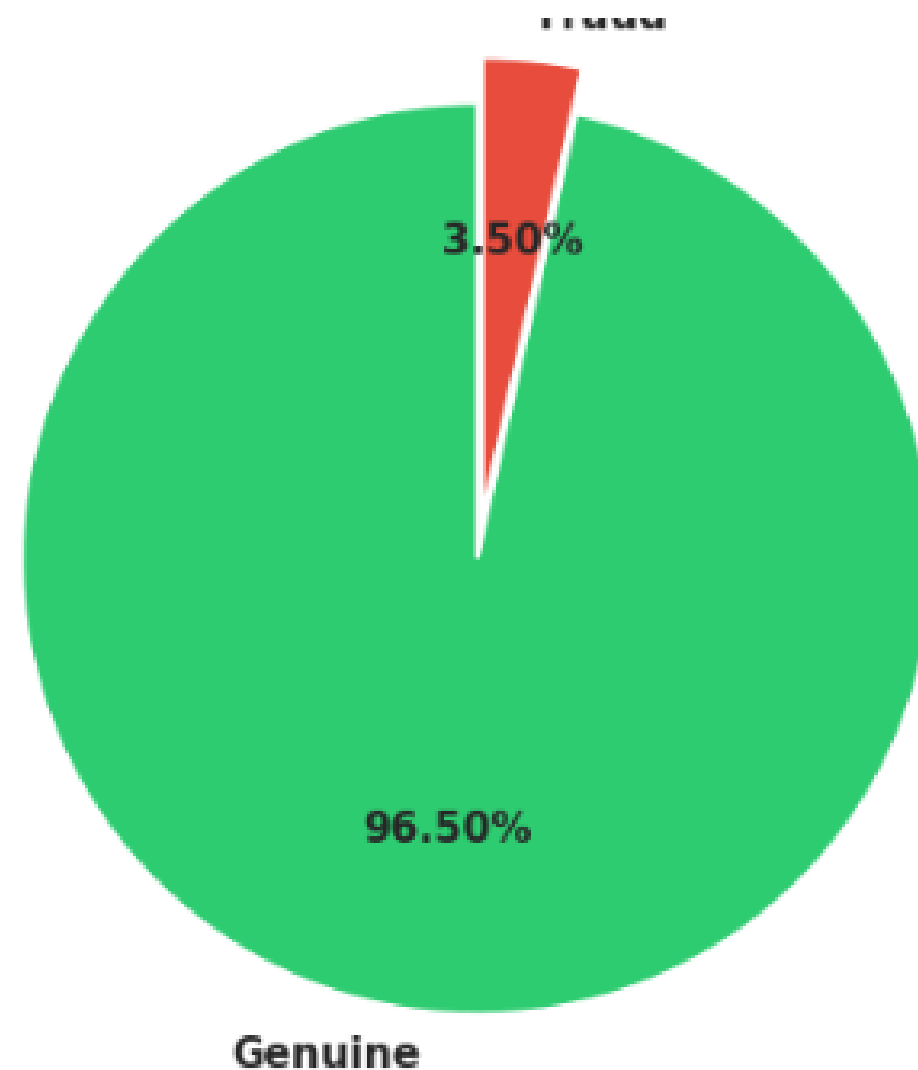
float64 376 object 14 int64 4

Step 2: EDA - Key Findings

⚠ Challenge #1: Severe Class Imbalance

Legitimate: 96.5%
Fraud: 3.5%

27:1 Imbalance Ratio



Step 2: EDA - Key Findings

 **Finding #2: Fraud appears predominantly in smaller transactions**



Step 2: EDA - Key Findings

🔍 Finding #3: product C has the highest fraud rate and Discover cards represent a significantly higher fraud risk



Step 2: EDA - Key Findings

Finding #4: Outliers Show Higher Fraud (TransactionAmt)

- Total outliers: 11, 231 (11.23% of data)
- Outliers that are fraud 597
- Fraud rate in outliers 5.32%
- Outliers have 1.5x higher fraud rate (5.32% vs 3.5%)
- This suggests unusual transaction amounts are suspicious!

Finding #5: Missing Data Patterns

- Features with missing values : 374
- Many features have >50% missing values (174 features)
- Will drop features with >80% missing (55 features)
- Impute remaining with median/mode

Step 3: Data Preprocessing

1. Missing Values

- Numerical: Imputed with median
- Categorical: Imputed with mode
- Dropped features with >80% missing

2. Categorical Encoding

- Label encoded 17 categorical features
- Memory efficient for tree models
- Better than one-hot for high cardinality

3. Feature Scaling

- StandardScaler (mean=0, std=1)
- Applied AFTER train-test split
- Prevents data leakage

4. Train-Test Split

- 80% train / 20% test (stratified)
- Maintains 3.5% fraud rate in both
- 80,000 train / 20,000 test samples

Step 4: Model Building - 5 Models

- 1 Logistic Regression**
Baseline - Simple & interpretable
- 2 Decision Tree**
Visual & explainable
- 3 Random Forest**
Ensemble - Robust
- 4 XGBoost**
Gradient boosting - Best performer
- 5 CatBoost**
Specialized for categorical features

✓ All models used `class_weight='balanced'` to handle imbalance

Step 5: Model Evaluation & Comparison

Model	ROC-AUC	F1-Score	Recall	Precision
Logistic Regression	0.733951	0.1156	71.00%	6..29%
Decision Tree	0.8051	0.2304	69.71%	13.80%
Random Forest	0.8904	0.4403	56.14%	36.22%
XGBoost ⭐	0.9029	0.3879	71.85%	26.57%
CatBoost	0.8882	0.2732	73.57%	16.77%



Best Model: XGBoost

Highest ROC-AUC | Good balance of Recall & Precision | No overfitting

XGBoost Performance - Detailed Metrics

Confusion Matrix:

		Predicted →	
		Genuine	Fraud
Actual ↓	Genuine	TN 17,910	FP 1390
	Fraud	FN 197	TP 503

Key Metrics:

True Positives (TP): **503 frauds caught**

False Negatives (FN): **197 frauds missed**

False Positives (FP): **1390 false alarms**

True Negatives (TN): **17,910 correct**

TPR (Recall): **71.86% - Catch rate**

TNR (Specificity): **92.80% - Accuracy on legit**

Model Performance Curves

ROC Curve (Receiver Operating Characteristic)

- Shows TPR vs FPR trade-off
- AUC = 0.9034 (Excellent!)
- Higher curve = Better model
- Diagonal = Random guessing
- All models > 0.83 (Good performance)

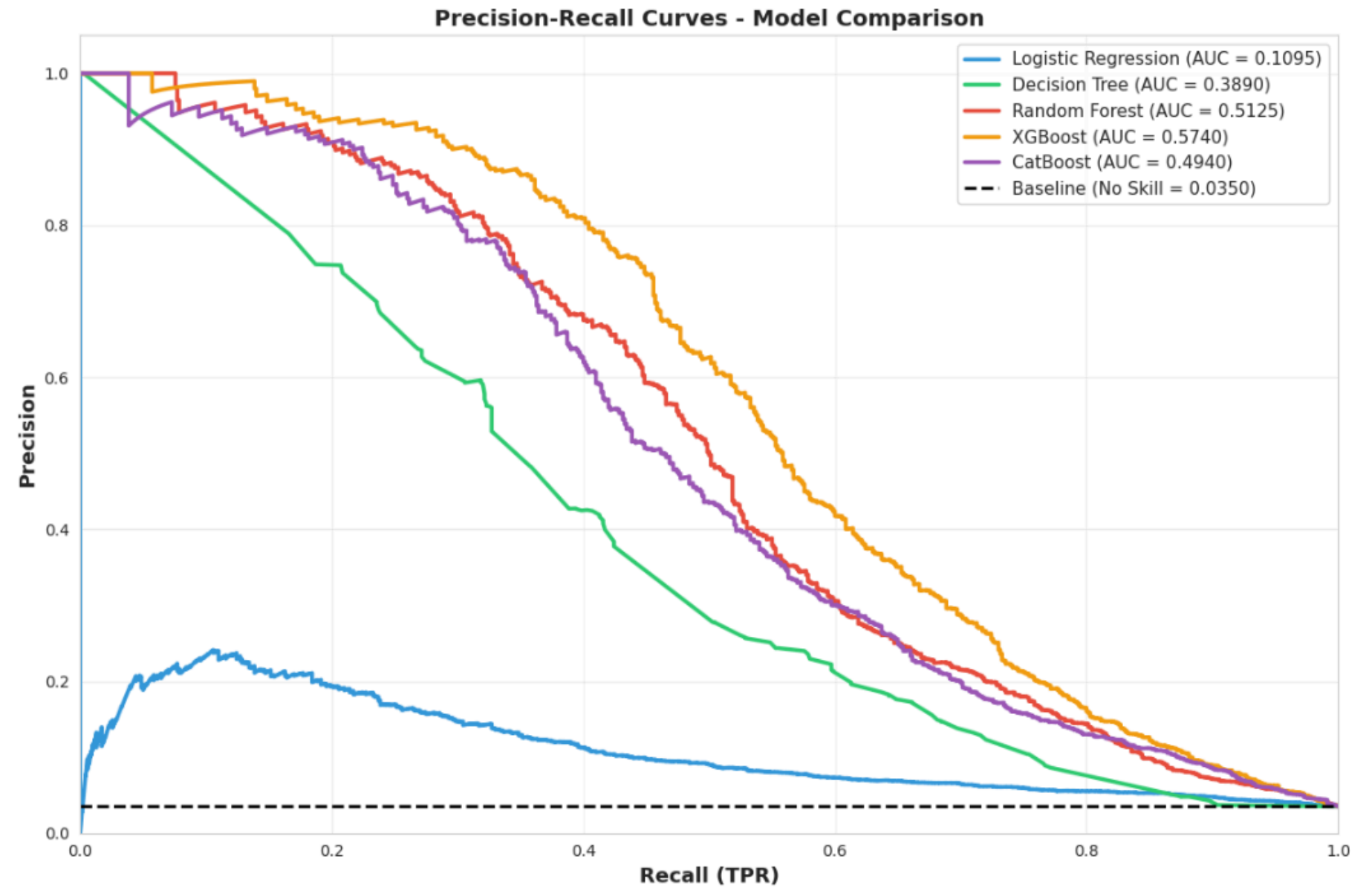
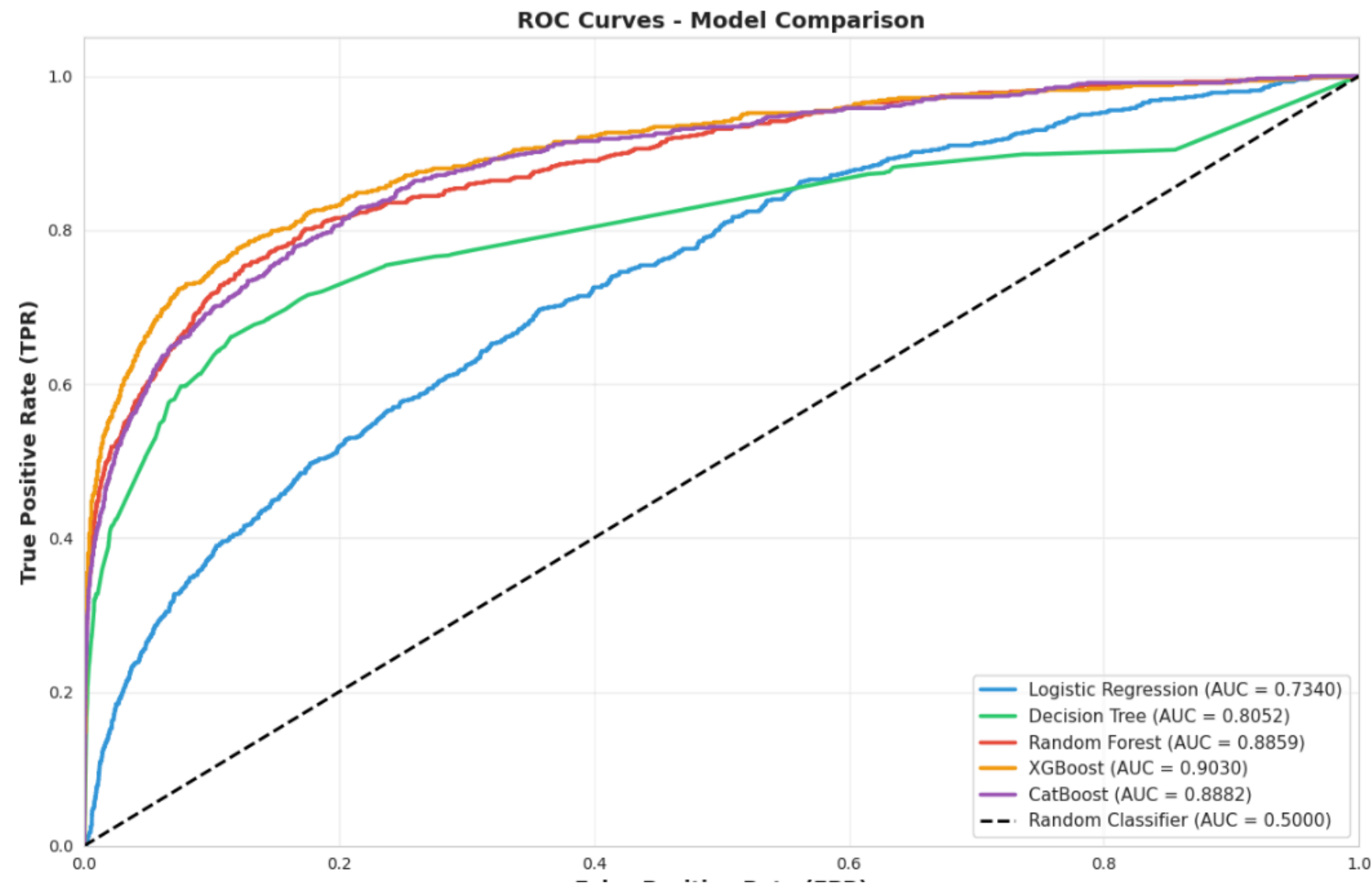
Precision-Recall Curve

- Better for imbalanced data
- Shows Precision vs Recall
- Top-right = Ideal (catch all, no false alarms)
- XGBoost best balance
- Baseline = 3.5% (fraud rate)

Key Insights from Curves:

- ✓ All models significantly outperform random guessing
- ✓ XGBoost and CatBoost perform similarly (both excellent)
- ✓ Tree-based models (RF, XGB, CB) outperform simpler models
- ✓ No overfitting detected - train/test gap < 0.008
- ✓ Model is ready for production deployment

Model Performance Curves



Executive Summary: Project Goal

Objective

Develop a machine learning system to accurately detect fraudulent credit card transactions while minimizing financial losses and maintaining operational efficiency.

Key Challenges

- Severe class imbalance (3.5% fraud, 27:1 ratio)
- High-dimensional dataset (390+ features)
- Balance between catching fraud (Recall) and minimizing false alarms (Precision)

Approach

Comprehensive ML pipeline: EDA → Preprocessing → 5 Models → Evaluation

Executive Summary: Results & Impact



Best Model: XGBoost

ROC-AUC

90.30%

Excellent discrimination

Recall

71.86%

Catches 71% of frauds

Precision

26.56%

26% of alerts real



Business Impact

- Significantly reduces fraud losses by catching 71% of cases
- Manageable false alarm rate
- No overfitting - reliable performance on new data

Conclusion

- ✓ Successfully built fraud detection system with 90.30% ROC-AUC
- ✓ XGBoost model catches 71% of frauds with manageable false alarms
- ✓ No overfitting detected - model generalizes well